

RNNs

C. V Jawahar

IIIT-Hyderabad

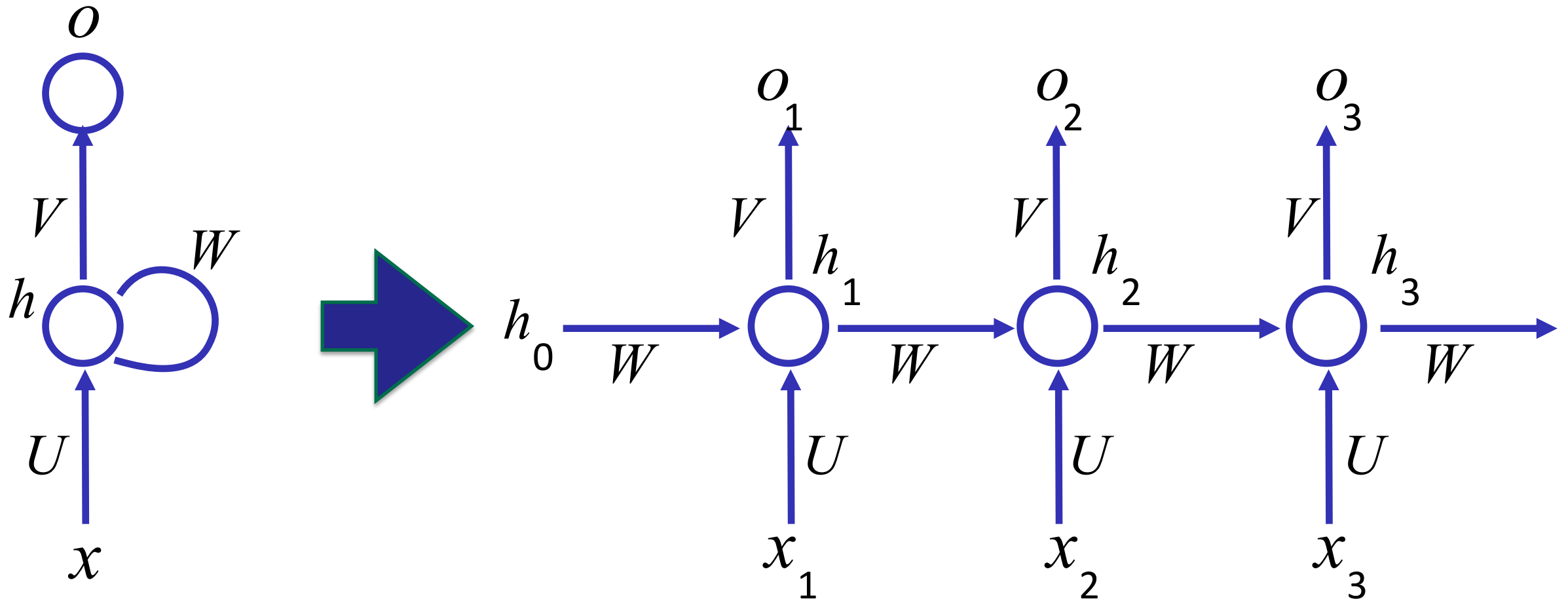
02 April 2026

Recap

CNNs

- 1D and 2D Convolutions
- Pad, Stride, Pool,
- Feature Maps
- Kernels, Windows,
- Weights, Parameters
- Design advancements
 - 1 X 1 Convolutions
 - Residual Connections
 - Deeper still smaller networks
- Pre-Train
- Fine Tune
- What do CNNs Learn?
- How do CNNs Learn?
- What type of inputs maximize activations?
- Different Performance Metrics/Concerns
- Different backbones

RNN basic architecture



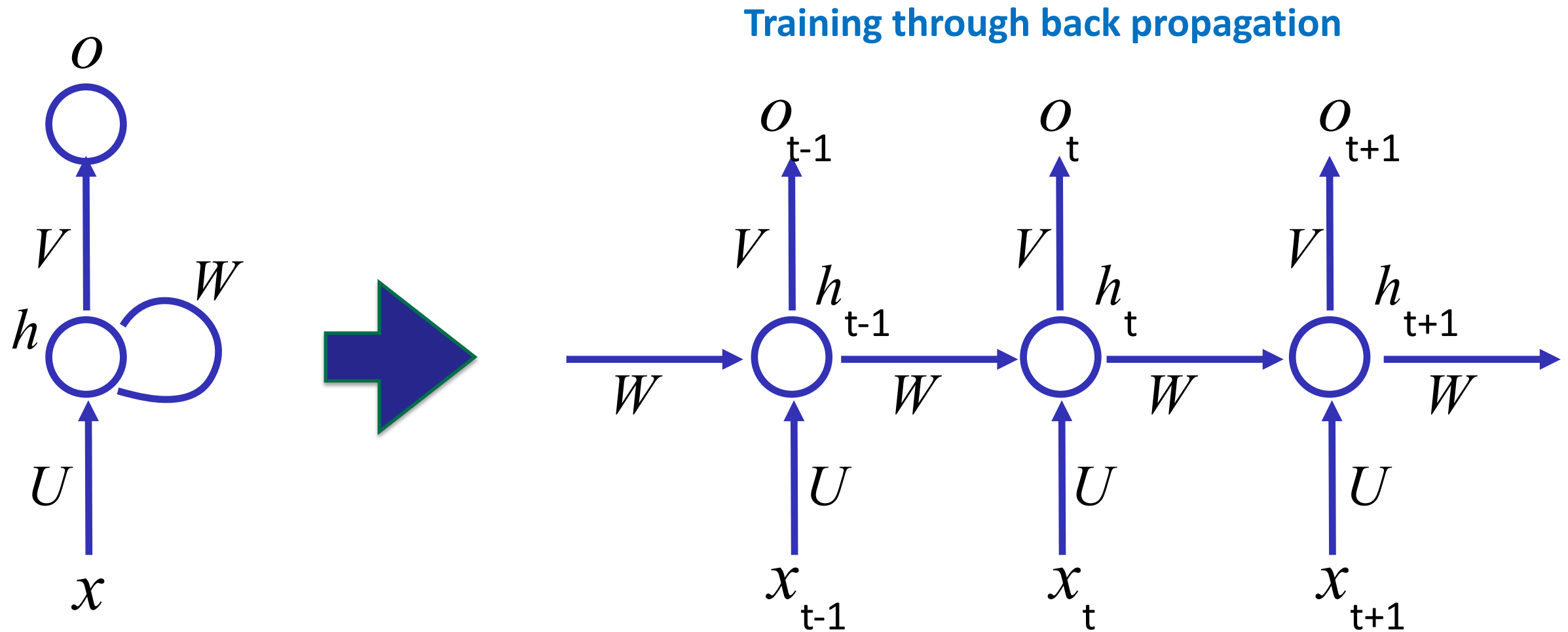
$$h_t = f(Ux_t + Wh_{t-1})$$

$$o_t = \text{softmax}_4(Vh_t)$$

RNN basic architecture

- x_t - input at time step t
- h_t - hidden state at time step t (memory of the network)
- o_t - output at time step t
- U, V, W are parameters (shared across all layers)

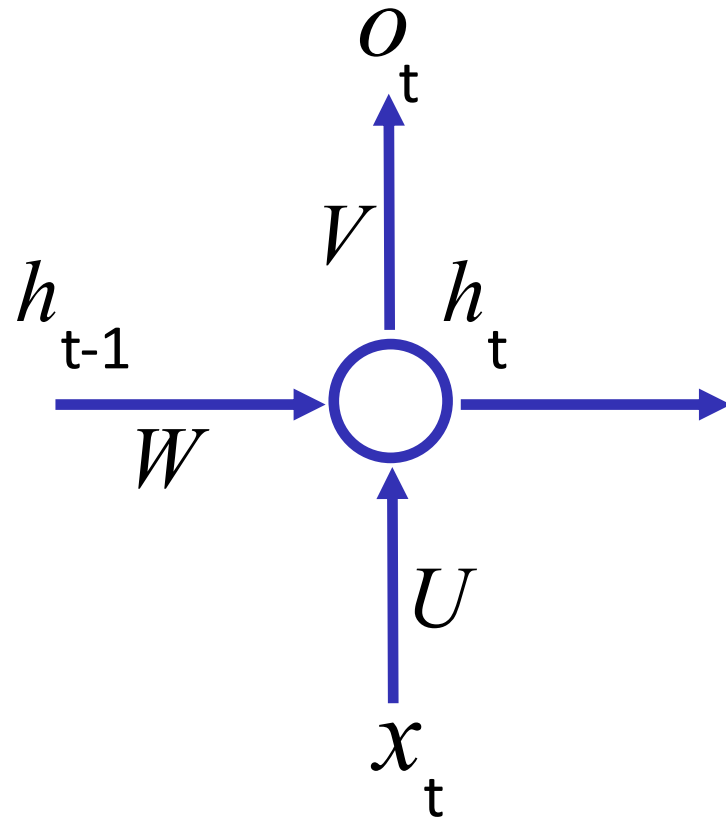
RNN basic architecture



$$h_t = f(Ux_t + Wh_{t-1})$$

$$o_t = \text{softmax}(Vh_t)$$

RNN basic architecture



$$h_t = f(Ux_t + Wh_{t-1})$$

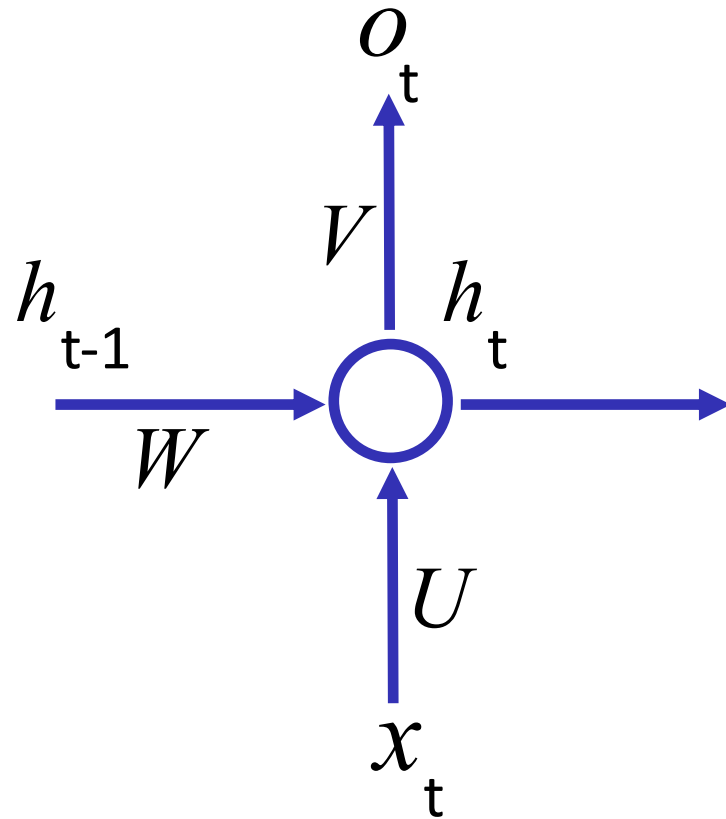
$$U = \begin{pmatrix} 0.5 & 0.4 \\ 0.4 & 0.6 \end{pmatrix}$$

$$W = \begin{pmatrix} 0.4 & 0.5 \\ 0.3 & 0.5 \end{pmatrix} \quad x_t = \begin{pmatrix} 0.4 \\ 0.2 \end{pmatrix} \quad h_{t-1} = \begin{pmatrix} 0.3 \\ 0.8 \end{pmatrix}$$

$$h_t = \tanh \left(\begin{matrix} U & x_t \\ \begin{pmatrix} 0.5 & 0.4 \\ 0.4 & 0.6 \end{pmatrix} & \begin{pmatrix} 0.4 \\ 0.2 \end{pmatrix} \end{matrix} + \begin{matrix} W & h_{t-1} \\ \begin{pmatrix} 0.4 & 0.5 \\ 0.3 & 0.5 \end{pmatrix} & \begin{pmatrix} 0.3 \\ 0.8 \end{pmatrix} \end{matrix} \right)$$

$$h_t = \tanh \left(\begin{pmatrix} .28 \\ .28 \end{pmatrix} + \begin{pmatrix} .52 \\ .29 \end{pmatrix} \right) = \tanh \begin{pmatrix} .88 \\ .77 \end{pmatrix} = \begin{pmatrix} .66 \\ .64 \end{pmatrix}$$

RNN basic architecture



$$U = \begin{pmatrix} 0.5 & 0.4 \\ 0.4 & 0.6 \end{pmatrix}$$

$$W = \begin{pmatrix} 0.4 & 0.5 \\ 0.3 & 0.5 \end{pmatrix} \quad x_t = \begin{pmatrix} 0.4 \\ 0.2 \end{pmatrix} \quad h_{t-1} = \begin{pmatrix} 0.3 \\ 0.8 \end{pmatrix}$$

$$h_t = \begin{pmatrix} .66 \\ .64 \end{pmatrix} \quad V = \begin{pmatrix} 0.4 & 0.7 \\ 0.3 & 0.1 \end{pmatrix}$$

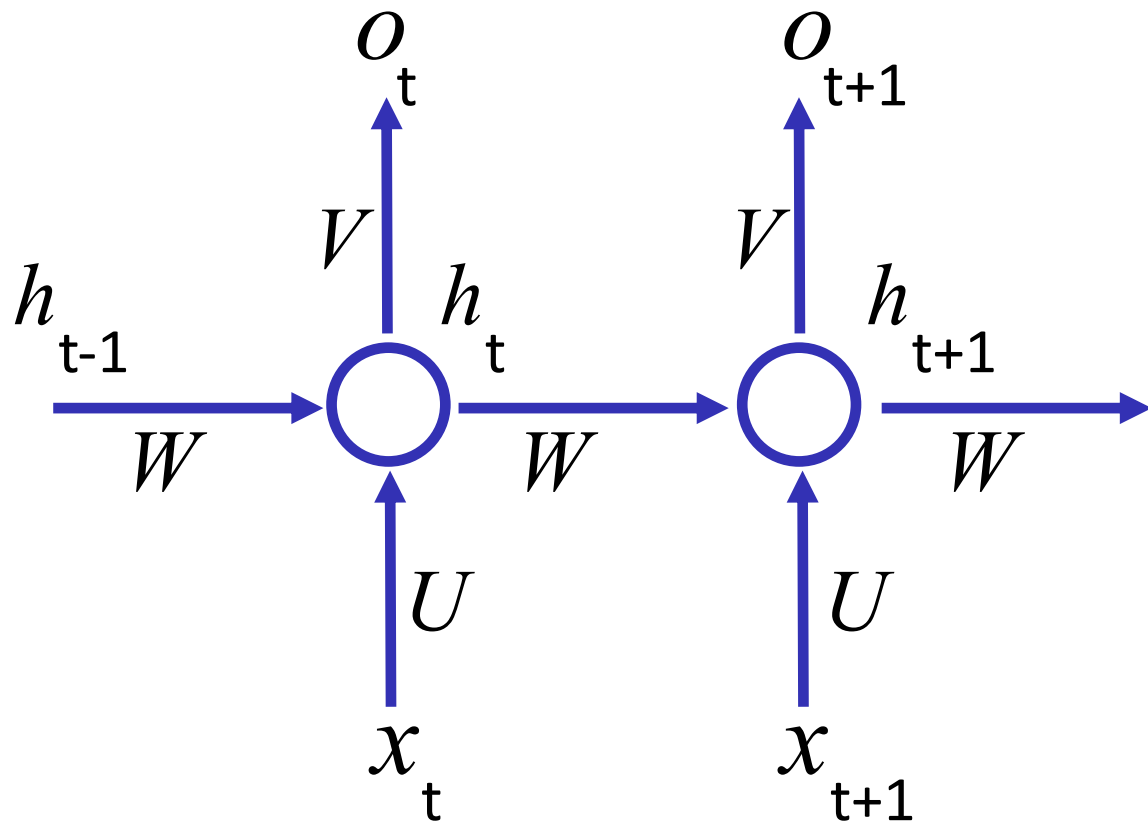
$$o_t = \text{softmax} \left(\begin{pmatrix} 0.4 & 0.7 \\ 0.3 & 0.1 \end{pmatrix} \begin{pmatrix} .66 \\ .64 \end{pmatrix} \right) = \begin{pmatrix} .61 \\ .39 \end{pmatrix}$$

$V \qquad h_t$

$$h_t = f(Ux_t + Wh_{t-1})$$

$$o_t = \text{softmax}(Vh_t)$$

RNN basic architecture



$$h_{t+1} = f(Ux_{t+1} + Wh_t)$$

$$U = \begin{pmatrix} 0.5 & 0.4 \\ 0.4 & 0.6 \end{pmatrix}$$

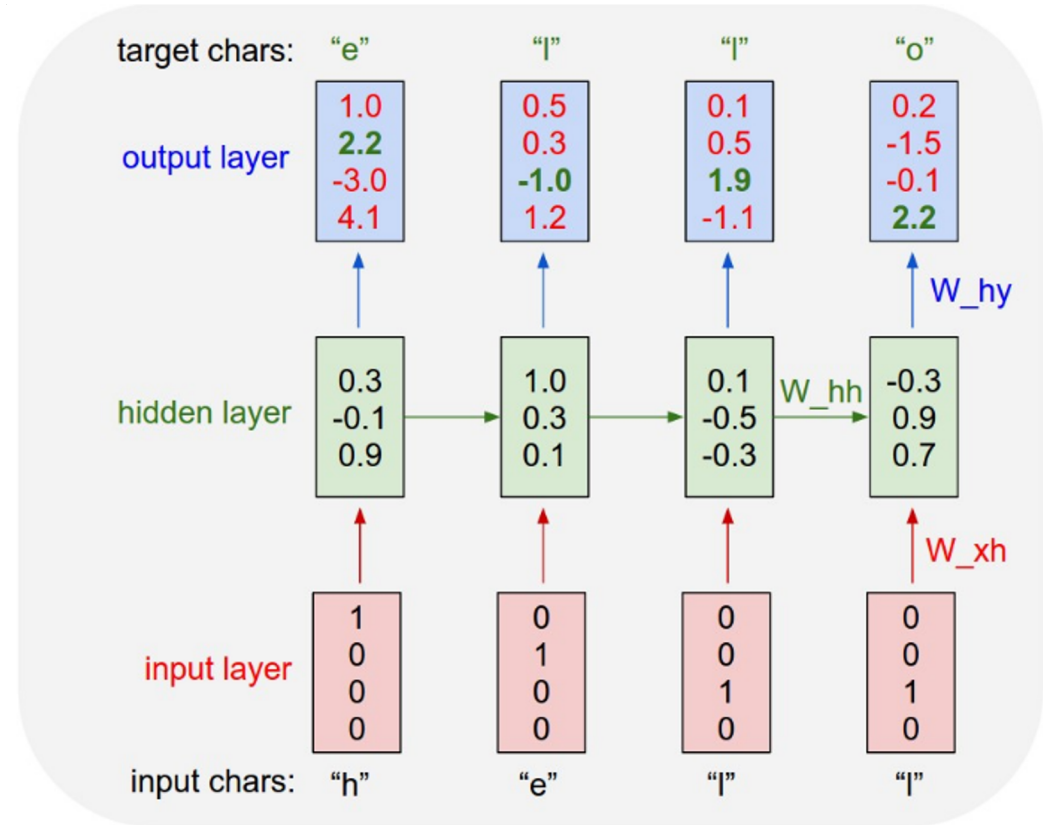
$$W = \begin{pmatrix} 0.4 & 0.5 \\ 0.3 & 0.5 \end{pmatrix} \quad x_{t+1} = \begin{pmatrix} 0.5 \\ 0.4 \end{pmatrix} \quad h_t = \begin{pmatrix} .66 \\ .64 \end{pmatrix}$$

$$h_{t+1} = \tanh \left(\begin{matrix} U & x_{t+1} \\ \begin{pmatrix} 0.5 & 0.4 \\ 0.4 & 0.6 \end{pmatrix} & \begin{pmatrix} 0.5 \\ 0.4 \end{pmatrix} \end{matrix} + \begin{matrix} W & h_t \\ \begin{pmatrix} 0.4 & 0.5 \\ 0.3 & 0.5 \end{pmatrix} & \begin{pmatrix} .66 \\ .64 \end{pmatrix} \end{matrix} \right)$$

Character Level Language Modelling

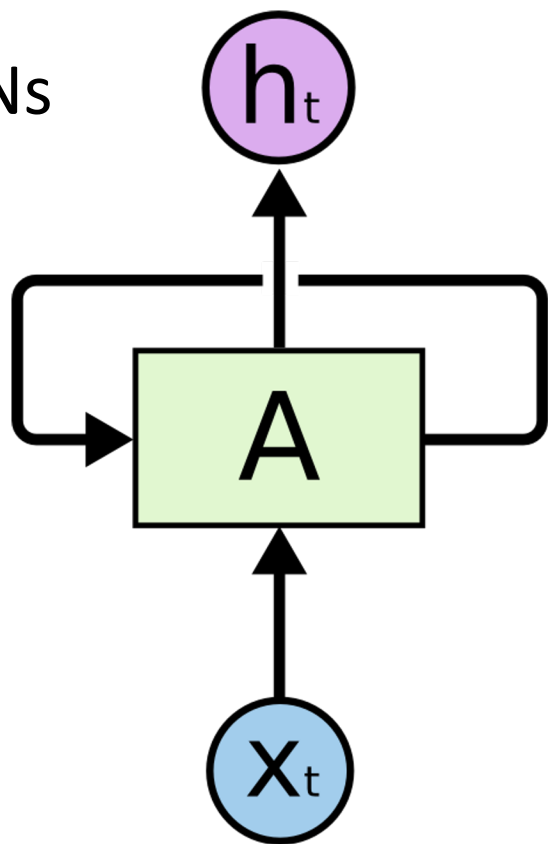
Task:

- Predicting the next character given the current character



Write Code

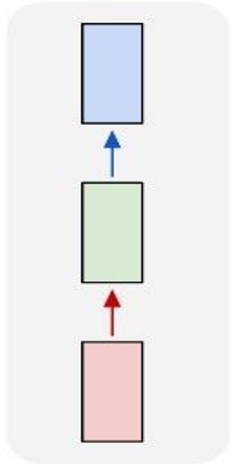
- RNNs



```
*  
* Increment the size file of the new incorrect UI_FILTER  
* of the size generatively.  
*/  
static int indicate_policy(void)  
  
int error;  
if (fd == MARN_EPT) {  
    /*  
    * The kernel blank will coeld it to userspace.  
    */  
    if (ss->segment < mem_total)  
        unblock_graph_and_set_blocked();  
    else  
        ret = 1;  
    goto bail;  
}  
segaddr = in_SB(in.addr);  
selector = seg / 16;  
setup_works = true;  
for (i = 0; i < blocks; i++) {  
    seq = buf[i++];  
    bpf = bd->bd.next + i * search;  
    if (fd) {  
        current = blocked;
```

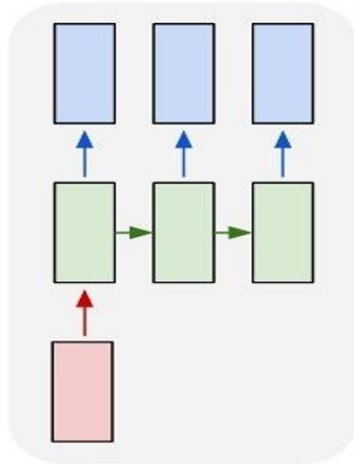
Recurrent Networks offer a lot of flexibility:

one to one



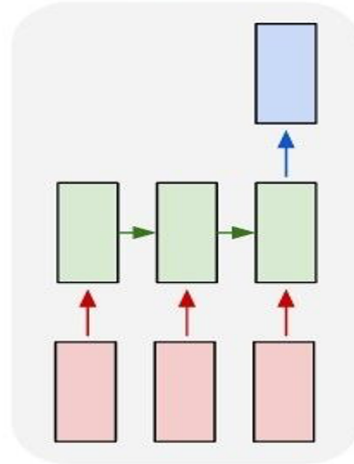
**vanilla neural
networks**

one to many



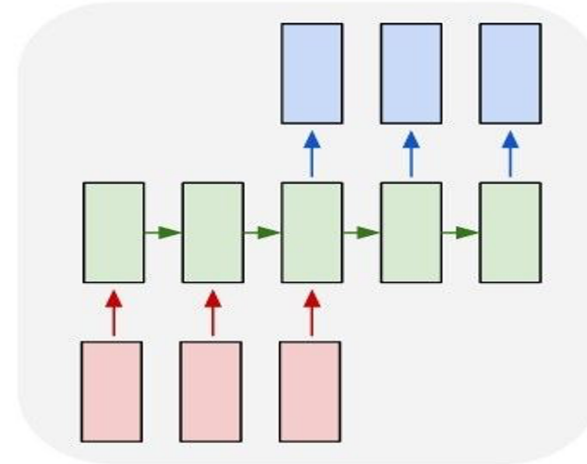
e.g. image captioning
image -> sequence of words

many to one



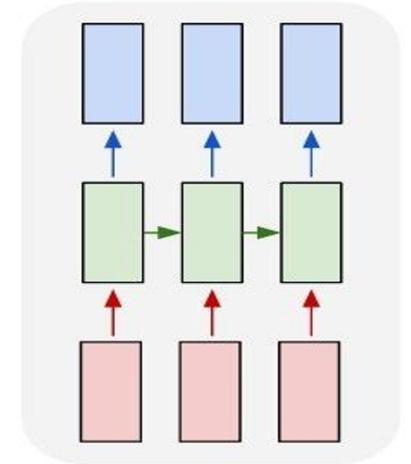
e.g. sentiment classification
sequence of words -> sentiment

many to many



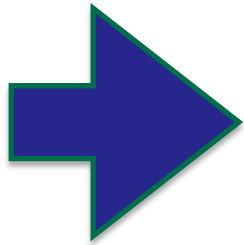
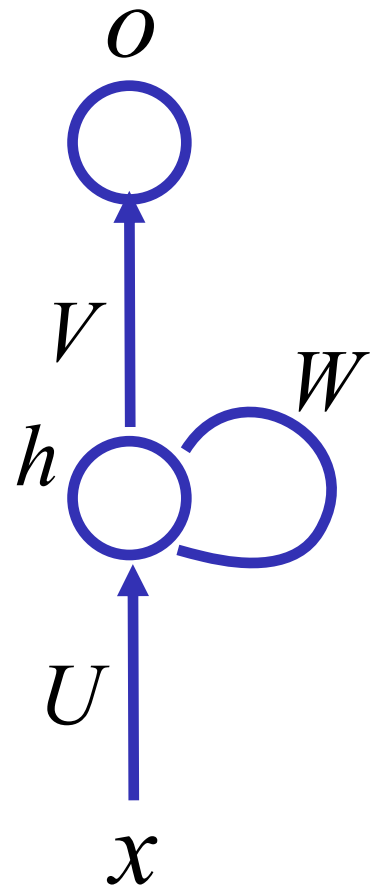
e.g. machine translation

many to many

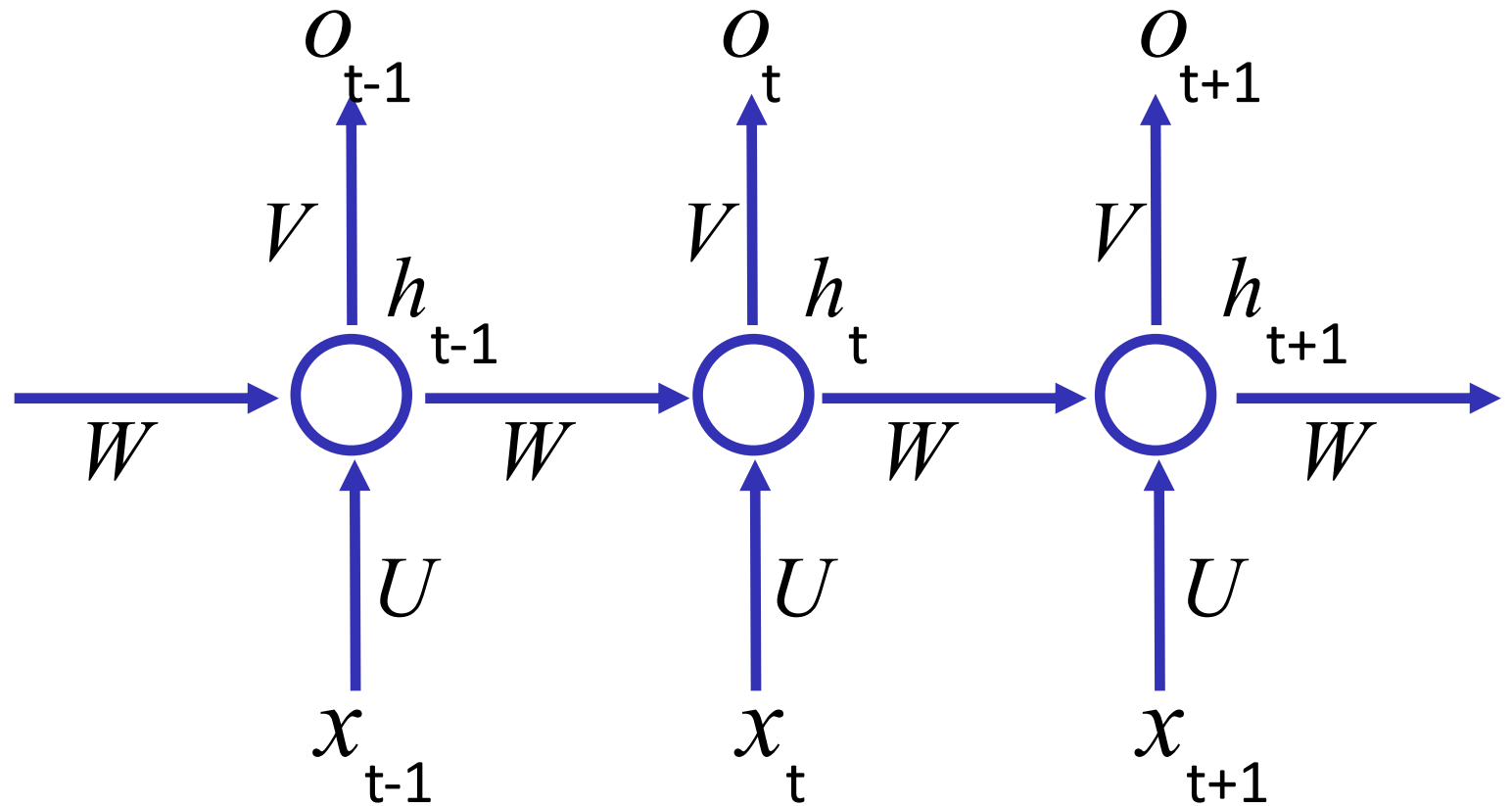


**e.g. video classification
on frame level**

RNN: Recap



Training through back propagation



$$h_t = f(Ux_t + Wh_{t-1})$$

$$o_t = \text{softmax}(Vh_t)$$

Back Propagation Through Time (BPTT)

$$\frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W}$$

$$\frac{\partial E_4}{\partial W} = \frac{\partial E_4}{\partial \hat{o}_4} \frac{\partial \hat{o}_4}{\partial h_4} \frac{\partial h_4}{\partial W}$$

But, $h_4 = f(Ux_3 + Wh_3)$

i.e., h_4 depends on W and h_3 ;
 h_3 depends on W and h_2 and so on..

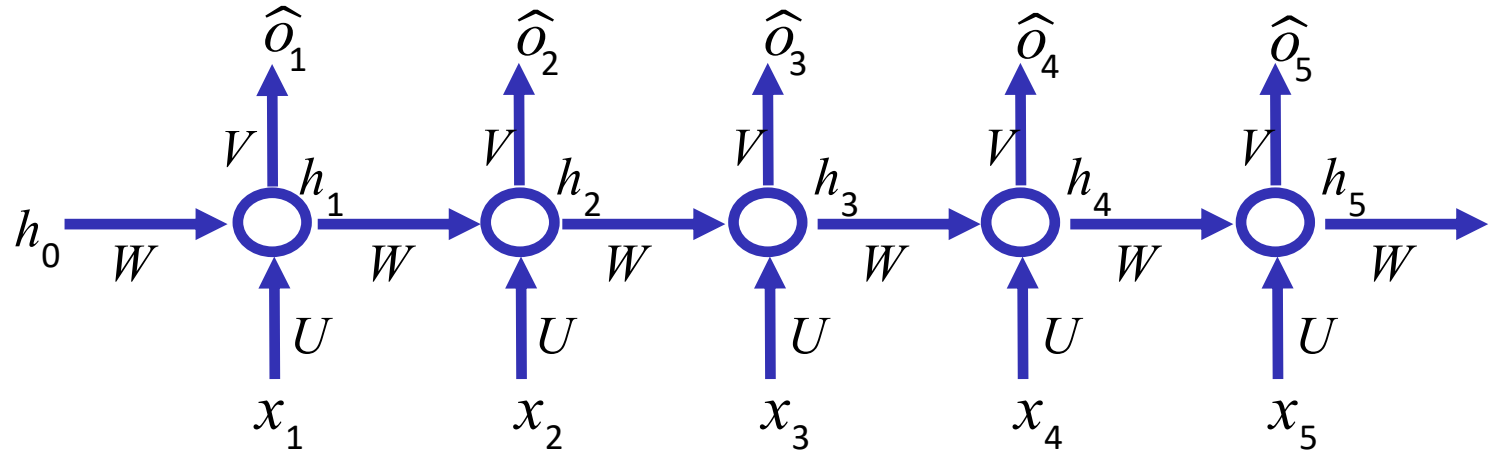
$$\frac{\partial E_4}{\partial W} = \sum_{k=1}^4 \frac{\partial E_4}{\partial \hat{o}_4} \frac{\partial \hat{o}_4}{\partial h_4} \frac{\partial h_4}{\partial h_k} \frac{\partial h_k}{\partial W}$$



$$\frac{\partial E_4}{\partial W} = \sum_{k=1}^4 \frac{\partial E_4}{\partial \hat{o}_4} \frac{\partial \hat{o}_4}{\partial h_4} \left(\prod_{j=k+1}^4 \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W}$$

This is our good-old chain rule.

[Do not worry about the exact equation]



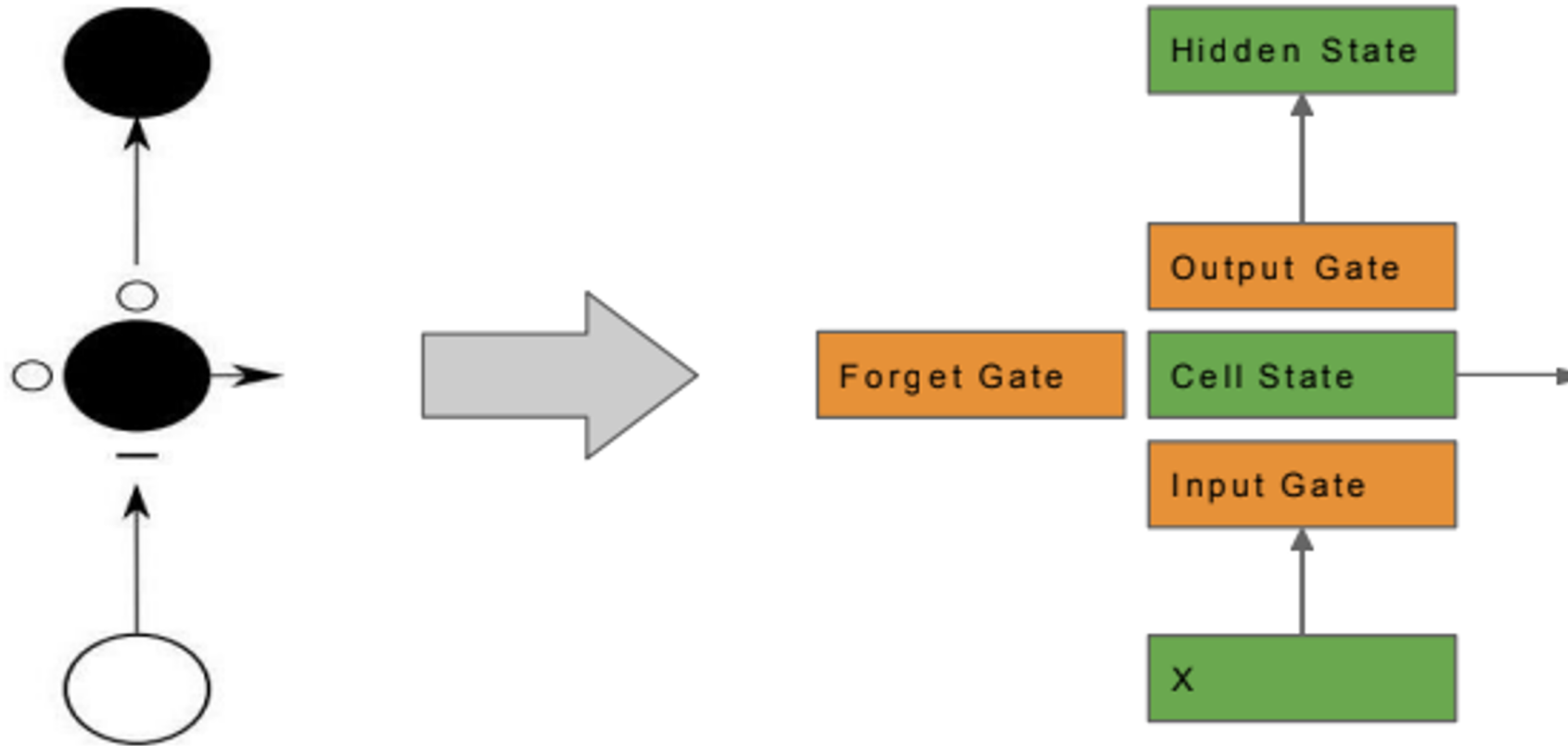
Vanishing Gradients

$$\frac{\partial E_4}{\partial W} = \sum_{k=1}^4 \frac{\partial E_4}{\partial \hat{o}_4} \frac{\partial \hat{o}_4}{\partial h_4} \frac{\partial h_4}{\partial h_k} \frac{\partial h_k}{\partial W}$$

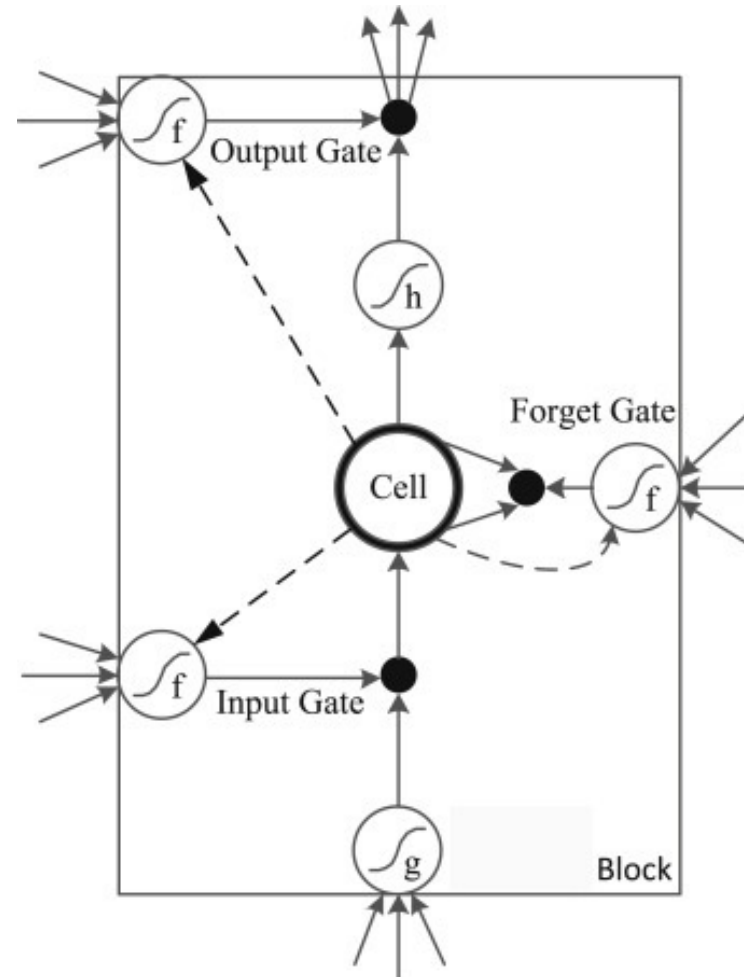
$$\frac{\partial E_4}{\partial W} = \sum_{k=1}^4 \frac{\partial E_4}{\partial \hat{o}_4} \frac{\partial \hat{o}_4}{\partial h_4} \left(\prod_{j=k+1}^4 \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_k}{\partial W}$$

- If a quantity is **less than 1.0**, then its continued multiplication can lead to **vanishing**
- If a quantity is **greater than 1.0**, then its continued multiplication can lead to **explosion**
- This is true for matrices as well.

LSTM Node

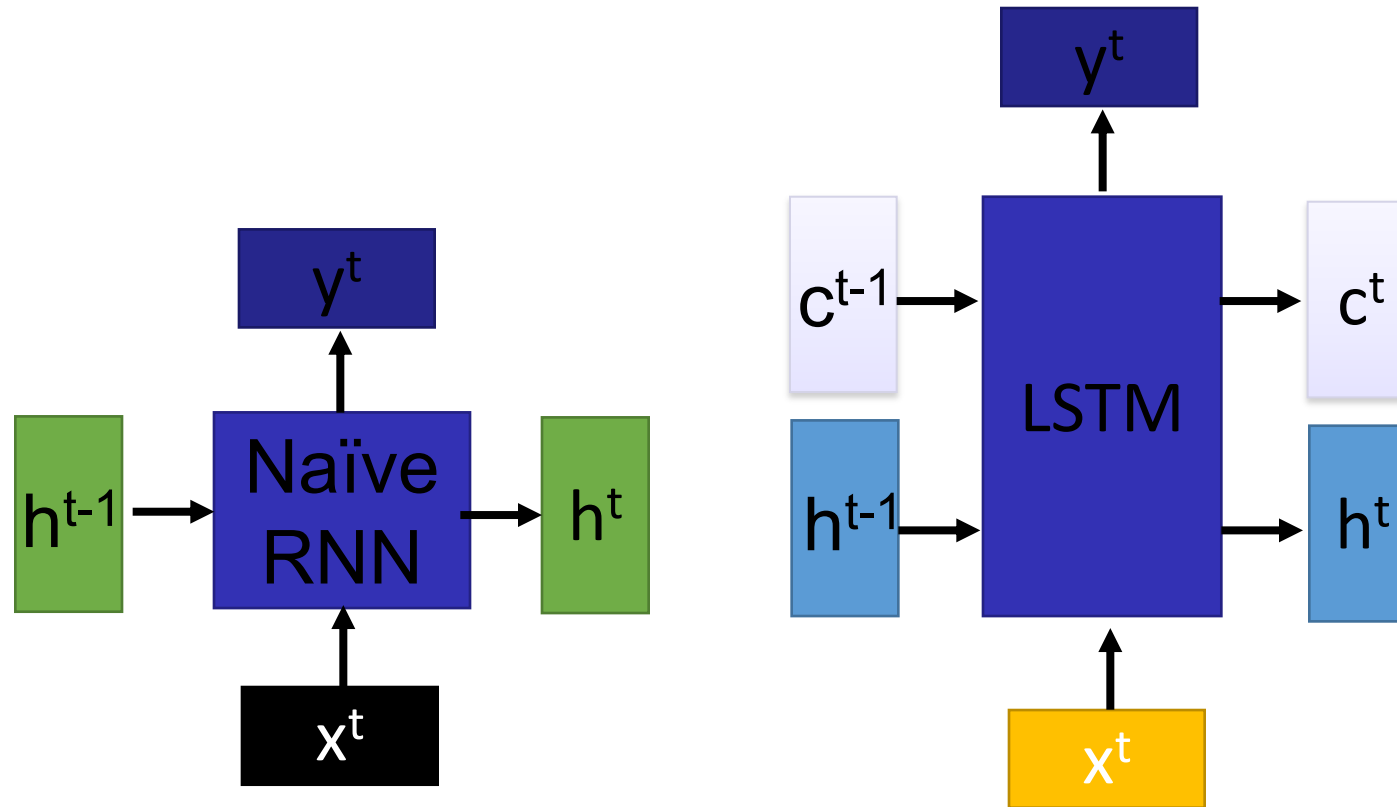


LSTM : Gates are “soft” controllable



Specific Equations are Avoided

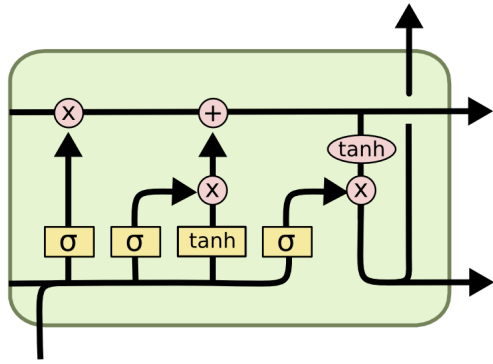
Naïve RNN vs LSTM



c changes slowly $\Rightarrow c^t$ is c^{t-1} added by something

h changes faster $\Rightarrow h^t$ and h^{t-1} can be very different

c^{t-1}



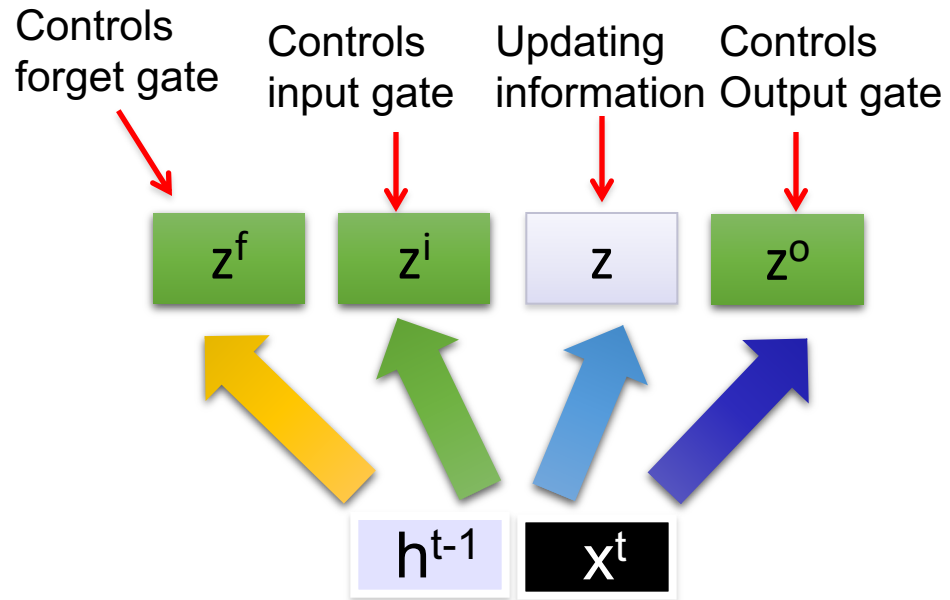
These 4 matrix computation should be done concurrently.

$$z = \tanh(W \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix})$$

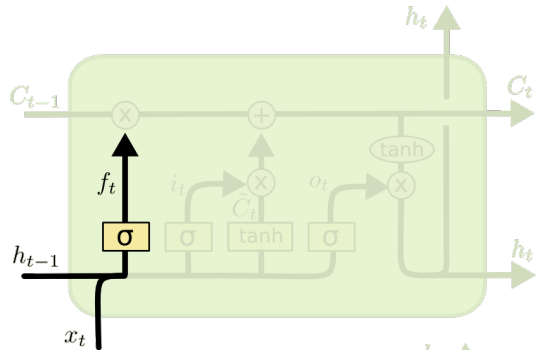
$$z^i = \sigma(W^i \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix})$$

$$z^f = \sigma(W^f \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix})$$

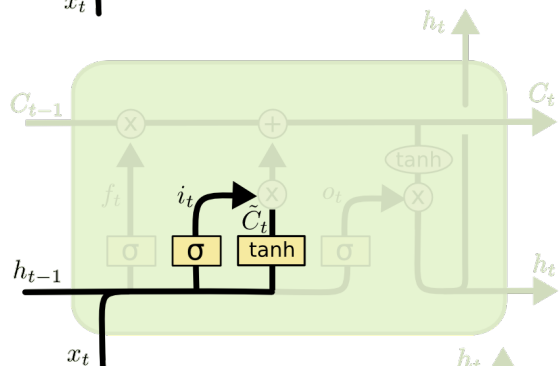
$$z^o = \sigma(W^o \begin{bmatrix} x^t \\ h^{t-1} \end{bmatrix})$$



Information flow of LSTM



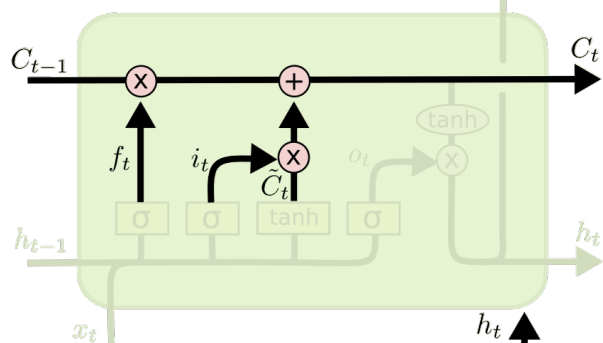
$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

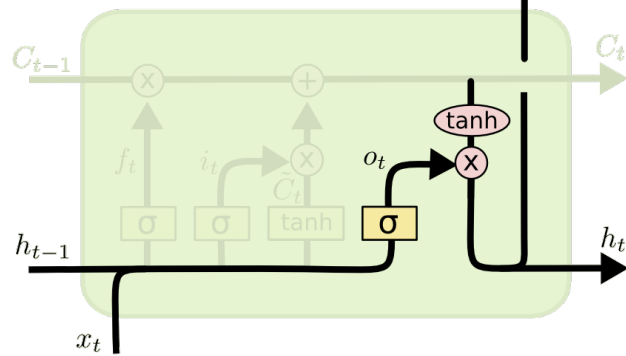
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

i_t decides what component
is to be updated.
 C'_t provides change contents



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

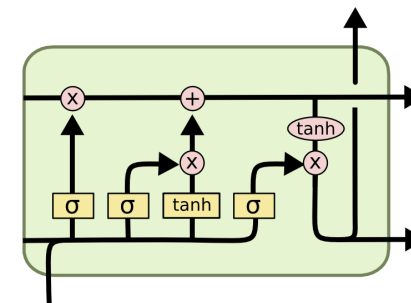
Updating the cell state



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Decide what part of the cell
state to output



Summary of Steps

Step 1: Decide How Much Past Data It Should Remember

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

f_t = forget gate
Decides which information to delete that is not important from previous time step

Step 2: Decide How Much This unit adds to the current state

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

i_t = input gate
Determines which information to let through based on its significance in the current time step

Step 3: Decide What part of the current state makes to the output.

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t * \tanh(C_t)$$

o_t = output gate
Allows the passed in information to impact the output in the current time step

Discussions

- RNNs: LSTM
- Training through BPTT
- Inference; Lack of parallelism?
- Character and other language modelling
- Smaller (compared to CNNs)
- Stacking multiple RNNs

Questions?